Projektübergabe: 4sight M365 Hardening — Kontext für eine neue Session

> **Zweck dieses Dokuments:** Du (Claude) übernimmst ein laufendes Projekt. Dieses Dokument ist dein vollständiger Kontext — Hintergrund, getroffene Entscheidungen, Architektur, aktueller Stand, Konventionen und offene Punkte. Lies es zuerst vollständig, bevor du etwas änderst. **Wichtig:** Die Weiterentwicklung des Codes soll mit **Claude Code** im Repository erfolgen (siehe Abschnitt „Arbeitsweise & Claude Code").

---

1. Worum geht es

Für **4sight-IT** (MSP) entsteht ein Werkzeugkasten für **Microsoft-365-Sicherheit bei Kunden mit Business-Premium-Lizenz** (Intune, Defender, Entra). Zwei Bausteine, die einen Kreis bilden:

1. **Assessment (read-only):** liest den Ist-Zustand eines Tenants aus und befüllt eine Sicherheits-Checkliste (Excel). Fertig und im Einsatz.
2. **Hardening Generator (aktive Entwicklung):** eine lokale Windows-Desktop-App (WPF), in der ein Techniker aus einer geführten Liste Einstellungen wählt; die App **erzeugt daraus ein PowerShell-Skript**, das diese Einstellungen setzt. Die App wendet selbst nichts an.

Kreislauf: **prüfen (Assessment) → setzen (Generator) → erneut prüfen.**

Zielnutzer sind **4sight-Techniker** (teils ohne tiefes M365-Wissen) → Oberfläche **streng geführt, sicher per Default**. Betrieb: **ein Tenant pro Durchlauf, lokal** (kein Web).

---

2. Die festen Leitplanken (nicht aufweichen)

Diese Entscheidungen sind mit dem Kunden abgestimmt und bindend:

- **Nur generieren, nie live anwenden** (die App erzeugt Skripte).
- **Keine Aussperr- UND keine Betriebs-Brecher** sind scharf wählbar. Risikomodell je Einstellung:
  - **Klasse A – sicher:** normal wählbar.
  - **Klasse B – betriebs-/sicherheitskritisch:** nur in **sicherer Form** wählbar (z. B. ASR/CFA/Network Protection nur **Audit**), gelb + Warnhinweis.
  - **Klasse C – aussperr-fähig** (z. B. Conditional Access scharf, Legacy-Auth-Block): in der UI **gesperrt/ausgegraut** und **vom Generator nie erzeugt** (doppelte Sperre).
- **Alle erzeugten Profile UNASSIGNED** — der scharfe Rollout (Zuweisung) ist ein bewusster, manueller Schritt des Technikers.
- **Break-Glass:** der Techniker trägt die Konten ein; das erzeugte Skript **verweigert die Ausführung durch ein Break-Glass-Konto**.
- **Least Privilege:** das Skript fordert nur die Graph-Scopes der gewählten Einstellungen an.
- **Feste Namenskonvention** (`4S-…`), **„aktualisieren statt doppelt anlegen"**.
- **Das Werkzeug besitzt seine eigenen `4S-…`-Profile** und **fasst fremde Profile nie an.**

---

3. Der Hardening Generator — Architektur

Stack: .NET 8 / **WPF** (`net8.0-windows`). Baut **nur unter Windows**. Leichtes MVVM.

Kernprinzip „Katalog als Daten": Jede wählbare Einstellung ist **eine JSON-Datei** im Ordner `catalog/` (Schema: `catalog/catalog.schema.json`). Inhalt (und damit die Sicherheit) ist von der App-Logik getrennt — **eine neue Einstellung = eine neue JSON, kein Code-Change.**

Datenfluss:

```
catalog/*.json  →  CatalogLoader  →  UI (A/B/C-Zustände)
                                  →  ScriptGenerator  →  erzeugtes PowerShell-Skript (.ps1)
```

Projektdateien: - `HardeningGenerator.csproj`, `nuget.config` (setzt nuget.org als Quelle — nötig für restore) - `App.xaml(.cs)`, `MainWindow.xaml(.cs)` (UI + „Skript erzeugen"-Knopf, SaveFileDialog, UTF-8-BOM) -

`Models/CatalogEntry.cs` (Abbild des Schemas) - `Services/CatalogLoader.cs` (lädt `catalog/*.json`) - `Services/ScriptGenerator.cs` (**Herzstück** — erzeugt das PowerShell-Skript) - `ViewModels/MainViewModel.cs`, `SettingItemViewModel.cs` - `tools/Export-IntuneConfigPolicy.ps1` (read-only Helfer, s. u.)

Das erzeugte Skript (Logik in `ScriptGenerator.cs` → `LogicBlock`): - Modi `-Mode Preview` (Standard, read-only) / `Apply` / `Revert`. - `$Settings` **-Array** = ein Eintrag je zu erzeugendem Profil; jedes trägt ein `Checks` **-Array** (ein Readback je Einstellung: `settingDefinitionId` + `desiredValue`). - **Preview** zeigt pro Einstellung Ist/Soll und scannt alle Intune-Profile auf **„bereits erfüllt durch anderes Profil"**. **Apply** legt an (vorher Backup), **Revert** stellt aus Backup wieder her. - `-SkipSatisfied` überspringt ein Profil nur, wenn **alle** seine Einstellungen schon erfüllt sind. - **Break-Glass-Sperre**, **vereinte Scopes**, **unassigned**-Erzeugung. - **Re-Run = löschen+neu** (eine gesetzte Zuweisung geht dabei verloren — In-Place-Update ist offen).

ProfileGroup (Profil-Zusammenfassung): Katalogeinträge mit gleicher `profileGroup.id` werden in **ein** gemeinsames Profil zusammengeführt (deren `settings[]` vereint). Beispiel: die AV- Einträge teilen `defender-av` → ein Profil `4S-Defender-AV-Baseline`. Einträge ohne `profileGroup` bleiben eigenständig.

---

4. Katalog-Schema (Felder)

Siehe `catalog/catalog.schema.json`. Wichtigste Felder je Eintrag: `id`, `checklistNr`, `workload`, `title`, `tier`, `riskClass` **(A/B/C)**, `safeMode` (`assignment` = unassigned; `mode` = enforce/audit/reportOnly), `ui` (`selectable`, `locked`, `warnText`), `requiredScopes`, `naming.policyName`, optional `profileGroup` (`id`, `policyName`, `title`), `graph` (`resource`, `createBody` = parametrisierter Payload mit `{{policyName}}` / `{{timestamp}}`, `readback` = `settingDefinitionId` + `desiredValue`), `revert`, `verifyInTestTenant`.

---

5. Aktueller Katalog (10 Einträge)

Gruppe `defender-av` → ein Profil `4S-Defender-AV-Baseline` (7 Settings, Klasse A): B-08 Echtzeitschutz, B-09 Cloud-Schutz, B-10 PUA=Block, B-11 Verhaltensüberwachung, B-13a Skript-Scan, B-13b Archiv-Scan, B-13c Download-/Anlagen-Scan.

Eigenständige Profile (Klasse B, Audit): B-12 Network Protection (`4S-Defender-NetworkProtection-Audit`), B-17 Controlled Folder Access (`4S-Defender-ControlledFolderAccess-Audit`).

Gesperrt (Klasse C): A-04 „CA: MFA für alle" (`locked` — nie erzeugbar, Demo der UI-Sperre).

Alle Intune-Settings laufen über `https://graph.microsoft.com/beta/deviceManagement/configurationPolicies` (Settings Catalog, `technologies: "mdm,microsoftSense"`, Choice-Settings `_1` = Ein / `_2` = Audit).

---

6. Validierungsstand

- **Live im Test-Tenant geprüft (Apply + Revert erfolgreich):** der gesamte aktuelle 10-Einträge- Katalog, inkl. zusammengeführter AV-Baseline (7 Settings), Network Protection und Controlled Folder Access. Die „bereits erfüllt"-Erkennung und `-SkipSatisfied` funktionieren.
- **Nur hier (Cloud) prüfbar war:** PowerShell-Parser über jedes erzeugte Skript, JSON-/Schema- Validität, und die Objekt→Graph-JSON-Serialisierung. **Die WPF-App selbst wurde unter Windows erfolgreich gebaut (`dotnet build`).**

---

7. Offene Punkte / Roadmap

Unmittelbar als Nächstes: ASR-Regeln (B-16). ASR-Regeln haben im Settings Catalog **regelspezifische `settingDefinitionId`s** — die werden **nicht geraten**. Vorgehen: mit `tools/Export-IntuneConfigPolicy.ps1` die echte Struktur aus einem Test-Tenant exportieren (Techniker legt in Intune ein Settings-Catalog-Profil „TEMP-ASR- Export" mit ASR-Regeln auf **Audit** an, dann `-Name "TEMP-ASR-Export" -OutFile asr.json`) und daraus den Eintrag **B-16** bauen (Klasse B, Audit, eigenständiges Profil `4S-Defender-ASR-Audit`). → *Der nächste Arbeitsschritt wartet auf diese `asr.json`.*

Weiter: - Firewall (B-18/19) und BitLocker (B-21) — ebenfalls per Export (verschachtelte/Gruppen-Settings). - Cloud-Blockierebene (High) und Tamper Protection — per Export verifizieren, dann in die AV-Gruppe. - **In-Place-Update** statt löschen+neu (Zuweisung über Re-Runs erhalten). - Phase 2: **Entra** (CA **nur Report-only**, automatischer Break-Glass-Ausschluss), dann Defender/EOP, Exchange. - Komfort: Pakete/Bundles; **Rückschreiben in die Checkliste** nach dem Setzen; evtl. MSP/Multi-Tenant.

8. Konventionen (für alle erzeugten/gelieferten Skripte)

- **PowerShell 5.1 UND 7 kompatibel** — **keine** PS7-only-Syntax (`? :`, `??`). `if/else`-Zuweisungen sind ok.
- **UTF-8 mit BOM** speichern (sonst kaputte Umlaute in Windows PowerShell 5.1).
- **Jedes erzeugte Skript gegen den PowerShell-AST-Parser prüfen** (Syntax).
- **Graph-Setting-IDs im Test-Tenant verifizieren**; komplexe/verschachtelte IDs **nicht raten**, sondern exportieren.
- **Sicherheits-Invarianten nie aufweichen:** Klasse C nie erzeugen; Profile unassigned; Break-Glass verweigern; Backup vor jeder Änderung.

9. Verwandter Baustein: Assessment + Checkliste (fertig)

- `Invoke-M365SecurityAssessment.ps1` — **read-only**, verbindet Graph + Exchange Online + Security & Compliance + DNS, prüft **53 Kontrollpunkte** und schreibt Ist-Wert/Status je „Nr." in die Excel-Checkliste. Punkte ohne Auto-Check werden explizit „manuell"-markiert.
- `M365_BusinessPremium_Sicherheit_Checkliste_v1.2.xlsx` — die Checkliste (111 Punkte, Blätter „Checkliste", „Auslese-Mapping", „Anhang ASR-Regeln"). Der Generator-Katalog dockt über die Nr. an diese Checkliste an.
- Details: `README_Assessment.md`, `Anleitung_Checkliste_befuellen.md`.

10. Arbeitsweise & Claude Code (bitte beachten)

Der Code soll ab jetzt mit Claude Code im Git-Repository weiterentwickelt werden. Gründe: die WPF-App **baut nur unter Windows** (eine reine Chat-/Cloud-Session kann sie nicht kompilieren), und Entwicklung profitiert von `dotnet build` / `run`, direkten Dateiedits, Git und dem Ausführen der erzeugten Skripte/des Export-Helfers gegen echte Tenants.

Empfohlene Aufteilung: - **Claude Code (Repo auf Windows-Dev-Maschine):** C#-/XAML-Änderungen, `dotnet build` / `run`, Katalog-JSONs ergänzen, Export-Helfer und erzeugte Skripte gegen Test-Tenants laufen lassen, Git-Verlauf. → Hier passiert die eigentliche Weiterentwicklung. - **Diese Projekt-Session (Chat):** Design-/Architekturentscheidungen, Review der erzeugten Skripte, Planung, und **dieses Dokument als Projektwissen** behalten.

Erste Schritte in der neuen Umgebung: 1. Repo anlegen, aktuellen Stand committen (`HardeningGenerator/` + die Assessment-/Doku-Dateien). 2. `dotnet build` (ggf. `nuget.config` + passendes .NET-8-SDK/-Architektur beachten, siehe `README_App.md` → Troubleshooting NU1100). 3. Als nächste Aufgabe die **ASR-**`asr.json` exportieren und den Eintrag **B-16** bauen.

11. Dateimanifest (im Übergabepaket `4sight-M365-Hardening_Projektuebergabe.zip`)

```
PROJECT_HANDOVER.md / .pdf      ← dieses Dokument (zuerst lesen)

app/HardeningGenerator/         ← die WPF-App (hier wird weiterentwickelt)
  HardeningGenerator.csproj, nuget.config
  App.xaml(.cs), MainWindow.xaml(.cs)
  Models/CatalogEntry.cs
  Services/CatalogLoader.cs, Services/ScriptGenerator.cs   ← Herzstück
  ViewModels/MainViewModel.cs, SettingItemViewModel.cs
  catalog/*.json (10 Einträge) + catalog.schema.json        ← Katalog als Daten
  tools/Export-IntuneConfigPolicy.ps1                       ← Autoren-Helfer (read-only)
  generated-example/Set-M365Hardening_EXAMPLE.ps1           ← Beispiel-Ausgabe
  README_App.md / .pdf                                      ← Build, Bedienung, Troubleshooting

assessment/                     ← der read-only Prüf-Teil (fertig)
  Invoke-M365SecurityAssessment.ps1
  README_Assessment.md
  Anleitung_Checkliste_befuellen.md / .pdf

checklist/
  M365_BusinessPremium_Sicherheit_Checkliste_v1.2.xlsx      ← Checkliste (Andockpunkt)
```

```
docs/
  Konzept_Hardening_Generator.md / .pdf                    ← Konzept/Sicherheitsmodell
  README_B08.md / .pdf                                     ← B-08-Vertikale (erste Durchstich-Doku)
  Set-M365Hardening_B-08.ps1                               ← eigenständiges B-08-Beispielskript
```

Nicht enthalten (bewusst, veraltet/ersetzt): der alte Word-Vordruck und die v1.0-Checkliste.